

Statistics Programs Record

Experiment 1: Finding Mean Using Python

Aim:

To find the mean of the given data.

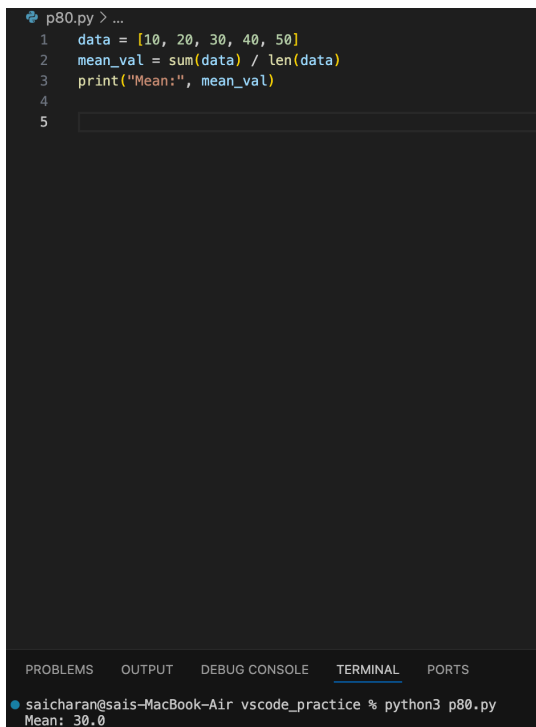
Algorithm:

1. Start
2. Create a list of numbers
3. Find the sum of elements
4. Divide by total number of elements
5. Display the mean

Code:

```
data = [10, 20, 30, 40, 50]
mean_val = sum(data) / len(data)
print("Mean:", mean_val)
```

Output:



The screenshot shows a VS Code editor with a Python file named p80.py containing the following code:

```
1 data = [10, 20, 30, 40, 50]
2 mean_val = sum(data) / len(data)
3 print("Mean:", mean_val)
4
5
```

Below the editor, the terminal output shows the command executed and the result:

```
saicharan@sais-MacBook-Air vscode_practice % python3 p80.py
Mean: 30.0
```

Result:

The mean of the given data is calculated successfully.

Experiment 2: Finding Median Using Python

Aim:

To find the median of the given data.

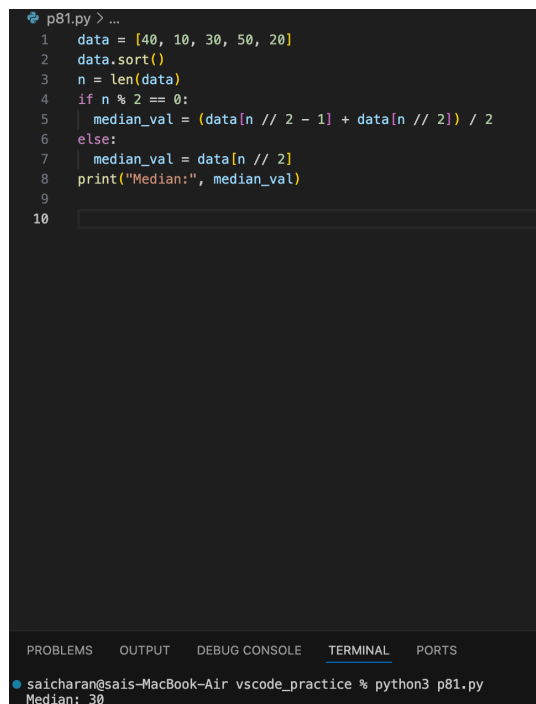
Algorithm:

1. Start
2. Create a list
3. Sort the list
4. Find the middle value
5. Display the median

Code:

```
data = [40, 10, 30, 50, 20]
data.sort()
n = len(data)
if n % 2 == 0:
    median_val = (data[n // 2 - 1] + data[n // 2]) / 2
else:
    median_val = data[n // 2]
print("Median:", median_val)
```

Output:

A screenshot of a code editor window showing a Python script named p81.py. The script defines a list 'data' with values [40, 10, 30, 50, 20], sorts it, and calculates the median. The terminal output at the bottom shows the command 'python3 p81.py' being executed and the output 'Median: 30'.

```
p81.py > ...
1  data = [40, 10, 30, 50, 20]
2  data.sort()
3  n = len(data)
4  if n % 2 == 0:
5      median_val = (data[n // 2 - 1] + data[n // 2]) / 2
6  else:
7      median_val = data[n // 2]
8  print("Median:", median_val)
9
10
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@sais-MacBook-Air vscode_practice % python3 p81.py
Median: 30

Result:

The median is calculated successfully.

Experiment 3: Finding Mode Using Python

Aim:

To find the mode of the given data.

Algorithm:

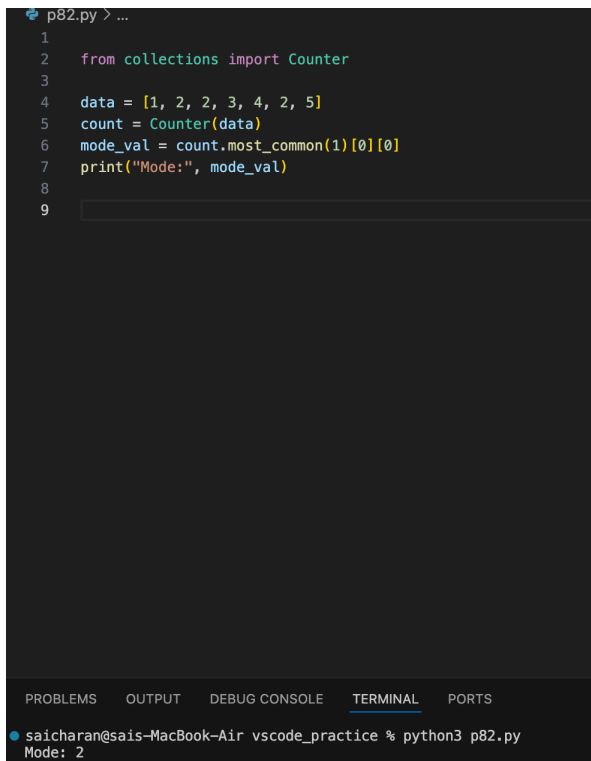
1. Import Counter
2. Create list
3. Count occurrences
4. Find most common value
5. Display mode

Code:

```
from collections import Counter

data = [1, 2, 2, 3, 4, 2, 5]
count = Counter(data)
mode_val = count.most_common(1)[0][0]
print("Mode:", mode_val)
```

Output:



The screenshot shows a VS Code editor with a file named p82.py. The code in the editor is as follows:

```
1
2 from collections import Counter
3
4 data = [1, 2, 2, 3, 4, 2, 5]
5 count = Counter(data)
6 mode_val = count.most_common(1)[0][0]
7 print("Mode:", mode_val)
8
9
```

Below the editor, the terminal window shows the command and output:

```
saicharan@sais-MacBook-Air vscode_practice % python3 p82.py
Mode: 2
```

Result:

The mode is calculated successfully.

Experiment 4: Finding Range Using Python

Aim:

To find the range of the data.

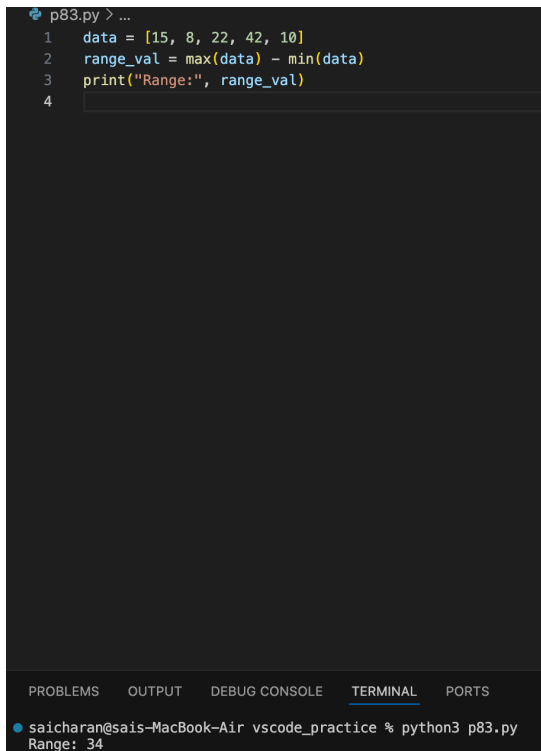
Algorithm:

1. Create list
2. Find maximum
3. Find minimum
4. Subtract minimum from maximum
5. Display range

Code:

```
data = [15, 8, 22, 42, 10]
range_val = max(data) - min(data)
print("Range:", range_val)
```

Output:



```
p83.py > ...
1 data = [15, 8, 22, 42, 10]
2 range_val = max(data) - min(data)
3 print("Range:", range_val)
4
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@sais-MacBook-Air vscode_practice % python3 p83.py
Range: 34

Result:

The range is calculated successfully.

Experiment 5: Finding Variance Using Python

Aim:

To find the population variance.

Algorithm:

1. Create list
2. Calculate mean
3. Calculate squared differences
4. Find average
5. Display variance

Code:

```
data = [2, 4, 4, 4, 5, 5, 7, 9]
mean_val = sum(data) / len(data)
variance_val = sum((x - mean_val) ** 2 for x in data) / len(data)
print("Variance:", variance_val)
```

Output:



The screenshot shows a VS Code editor with a file named 'p84.py'. The code in the editor is as follows:

```
1 data = [2, 4, 4, 4, 5, 5, 7, 9]
2 mean_val = sum(data) / len(data)
3 variance_val = sum((x - mean_val) ** 2 for x in data) / len(data)
4 print("Variance:", variance_val)
5
6
```

At the bottom, the 'TERMINAL' panel shows the command 'python3 p84.py' being executed, resulting in the output 'Variance: 4.0'.

Result:

The variance is calculated successfully.

Experiment 6: Finding Standard Deviation Using Python

Aim:

To find the standard deviation.

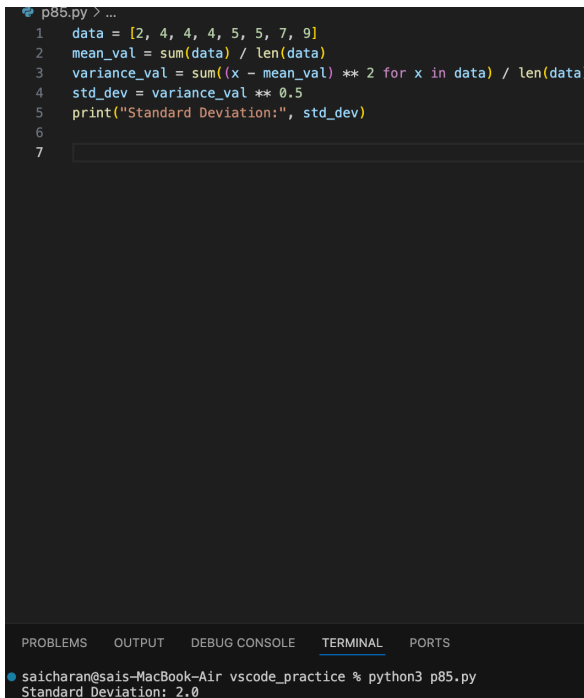
Algorithm:

1. Create list
2. Calculate mean
3. Find variance
4. Take square root
5. Display standard deviation

Code:

```
data = [2, 4, 4, 4, 5, 5, 7, 9]
mean_val = sum(data) / len(data)
variance_val = sum((x - mean_val) ** 2 for x in data) / len(data)
std_dev = variance_val ** 0.5
print("Standard Deviation:", std_dev)
```

Output:



The screenshot shows a VS Code editor with a file named 'p85.py' containing the following Python code:

```
1 data = [2, 4, 4, 4, 5, 5, 7, 9]
2 mean_val = sum(data) / len(data)
3 variance_val = sum((x - mean_val) ** 2 for x in data) / len(data)
4 std_dev = variance_val ** 0.5
5 print("Standard Deviation:", std_dev)
6
7
```

Below the editor, the 'TERMINAL' tab is active, showing the command 'python3 p85.py' and its output:

```
saicharan@sais-MacBook-Air vscode_practice % python3 p85.py
Standard Deviation: 2.0
```

Result:

The standard deviation is calculated successfully.

Experiment 7: Using statistics Module

Aim:

To calculate statistical measures using the statistics module.

Algorithm:

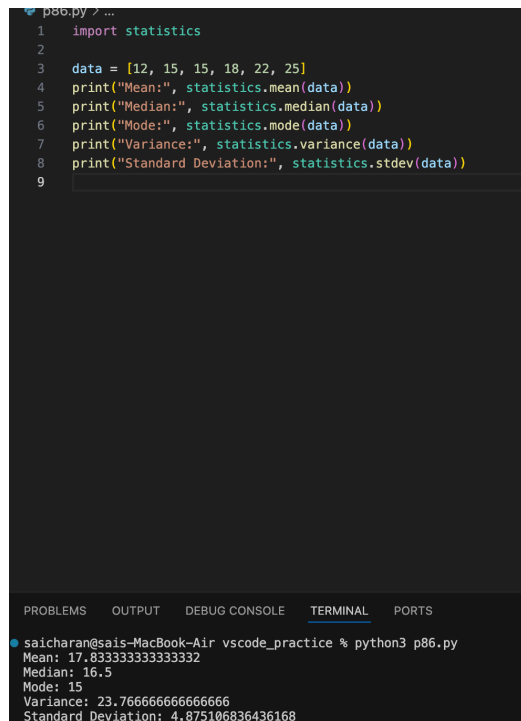
1. Import statistics
2. Create list
3. Calculate measures
4. Display output

Code:

```
import statistics

data = [12, 15, 15, 18, 22, 25]
print("Mean:", statistics.mean(data))
print("Median:", statistics.median(data))
print("Mode:", statistics.mode(data))
print("Variance:", statistics.variance(data))
print("Standard Deviation:", statistics.stdev(data))
```

Output:



The screenshot shows a code editor with a dark background. The code is written in Python and uses the statistics module to calculate various statistical measures for a list of numbers. The output is displayed in the terminal window at the bottom of the editor.

```
p86.py > ...
1  import statistics
2
3  data = [12, 15, 15, 18, 22, 25]
4  print("Mean:", statistics.mean(data))
5  print("Median:", statistics.median(data))
6  print("Mode:", statistics.mode(data))
7  print("Variance:", statistics.variance(data))
8  print("Standard Deviation:", statistics.stdev(data))
9
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@sais-MacBook-Air vscode_practice % python3 p86.py
Mean: 17.833333333333332
Median: 16.5
Mode: 15
Variance: 23.766666666666666
Standard Deviation: 4.875106836436168
```

Result:

Statistical measures are calculated successfully.

Experiment 8: Using NumPy Library

Aim:

To calculate statistical measures using NumPy.

Algorithm:

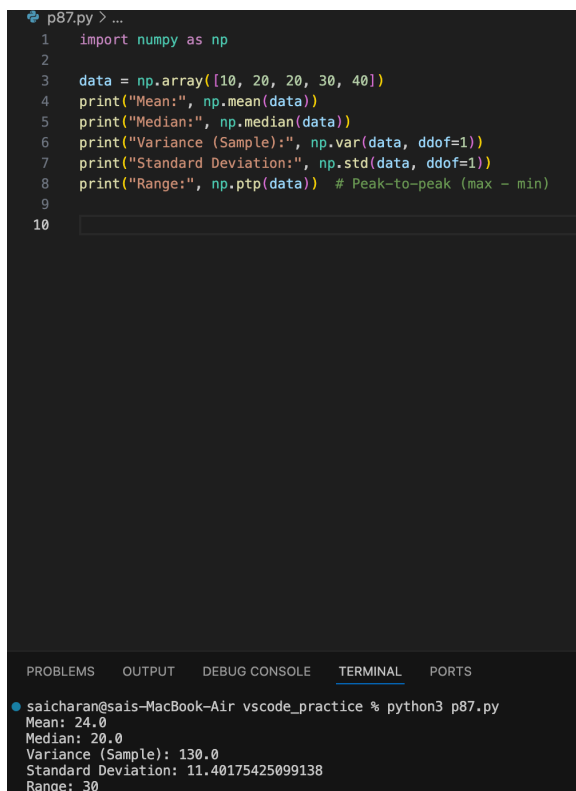
1. Import NumPy
2. Create array
3. Apply NumPy functions
4. Display results

Code:

```
import numpy as np

data = np.array([10, 20, 20, 30, 40])
print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Variance (Sample):", np.var(data, ddof=1))
print("Standard Deviation:", np.std(data, ddof=1))
print("Range:", np.ptp(data))
```

Output:



The screenshot shows a code editor with a Python script named p87.py. The script imports NumPy as np, creates an array [10, 20, 20, 30, 40], and prints various statistical measures. The terminal output at the bottom shows the results of these calculations.

```
p87.py > ...
1  import numpy as np
2
3  data = np.array([10, 20, 20, 30, 40])
4  print("Mean:", np.mean(data))
5  print("Median:", np.median(data))
6  print("Variance (Sample):", np.var(data, ddof=1))
7  print("Standard Deviation:", np.std(data, ddof=1))
8  print("Range:", np.ptp(data)) # Peak-to-peak (max - min)
9
10
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● saicharan@sais-MacBook-Air vscode_practice % python3 p87.py
Mean: 24.0
Median: 20.0
Variance (Sample): 130.0
Standard Deviation: 11.40175425099138
Range: 30
```

Result:

Statistical measures are calculated successfully using NumPy.

Experiment 9: Using SciPy for Mode

Aim:

To find the mode using SciPy.

Algorithm:

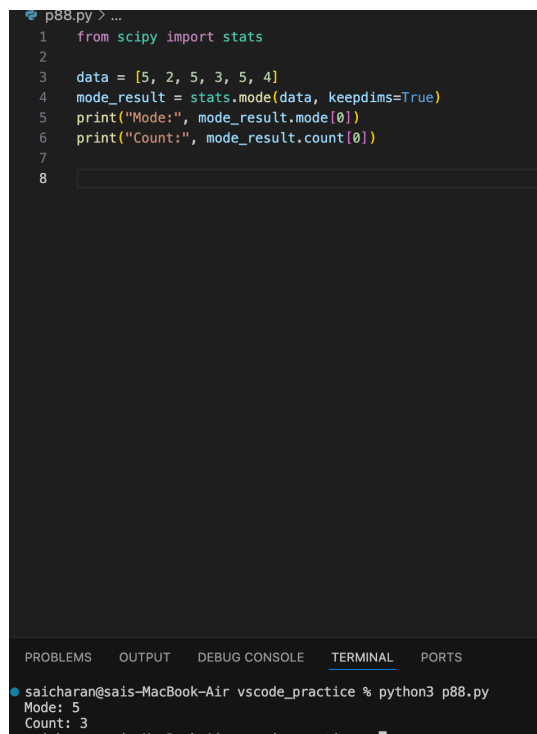
1. Import SciPy
2. Create list
3. Find mode
4. Display mode and count

Code:

```
from scipy import stats

data = [5, 2, 5, 3, 5, 4]
mode_result = stats.mode(data, keepdims=True)
print("Mode:", mode_result.mode[0])
print("Count:", mode_result.count[0])
```

Output:



The screenshot shows a VS Code editor window with a file named 'p88.py'. The code in the editor is as follows:

```
1 from scipy import stats
2
3 data = [5, 2, 5, 3, 5, 4]
4 mode_result = stats.mode(data, keepdims=True)
5 print("Mode:", mode_result.mode[0])
6 print("Count:", mode_result.count[0])
7
8
```

Below the editor, the 'TERMINAL' tab is active, showing the output of running the script:

```
saicharan@sais-MacBook-Air: vscode_practice % python3 p88.py
Mode: 5
Count: 3
```

Result:

Mode is calculated successfully using SciPy.

Experiment 10: Using Pandas for Complete Summary

Aim:

To calculate summary statistics using Pandas.

Algorithm:

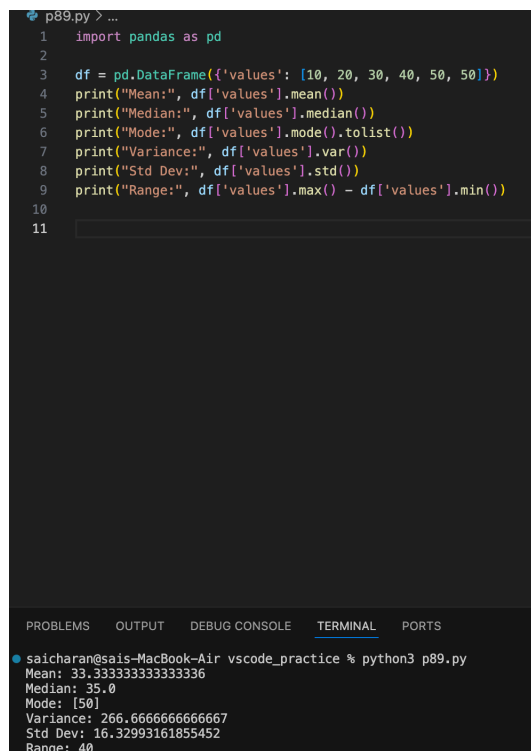
1. Import pandas
2. Create DataFrame
3. Calculate statistical measures
4. Display results

Code:

```
import pandas as pd

df = pd.DataFrame({'values': [10, 20, 30, 40, 50, 50]})
print("Mean:", df['values'].mean())
print("Median:", df['values'].median())
print("Mode:", df['values'].mode().tolist())
print("Variance:", df['values'].var())
print("Std Dev:", df['values'].std())
print("Range:", df['values'].max() - df['values'].min())
```

Output:



```
p89.py > ...
1  import pandas as pd
2
3  df = pd.DataFrame({'values': [10, 20, 30, 40, 50, 50]})
4  print("Mean:", df['values'].mean())
5  print("Median:", df['values'].median())
6  print("Mode:", df['values'].mode().tolist())
7  print("Variance:", df['values'].var())
8  print("Std Dev:", df['values'].std())
9  print("Range:", df['values'].max() - df['values'].min())
10
11
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
saicharan@sais-MacBook-Air vscode_practice % python3 p89.py
Mean: 33.333333333333336
Median: 35.0
Mode: [50]
Variance: 266.6666666666667
Std Dev: 16.32993161855452
Range: 40
```

Result:

Summary statistics are calculated successfully using Pandas.